

A Compressive Sensing Inspired Monte-Carlo Method for Combinatorial Optimization

Baptiste Chevalier^{1,*}, Shimpei Yamaguchi¹, Wojciech Roga¹, and Masahiro Takeoka^{1,2}

¹Department of Electronics and Electrical Engineering, Keio University, Yokohama 223-8522, Japan

²Advanced ICT Research Institute, National Institute of Information and Communications Technology (NICT), Koganei, Tokyo 184-8795, Japan

*chevalier.baptiste@keio.jp

ABSTRACT

In this paper, we present a Monte-Carlo Compressive Optimization algorithm, a new method to tackle combinatorial optimization problems, including Black-Box or complicated objective functions. The method relies on random queries to the objective function in order to estimate generalized moments. Next, a greedy algorithm from compressive sensing is repurposed to find the global optimum when not overfitting to the samples. We provide numerical results giving evidence that our method is competitive by comparing it with dual annealing. Moreover, we give theoretical justification for the success of the algorithm and analyze its properties. The practicality of our algorithm is enhanced by the ability to tune the heuristic parameters to the available computational resources. An end-to-end open-source implementation is available to use our method.

Introduction

Optimization problems lie in the very core of many important modern world applications, including engineering, finance, and physics. However, finding efficient ways to solve these problems remains an important concern, since computational time usually increases quickly with the size of the instance. For example, minimization of the cost function in neural networks over the space of network parameters becomes harder as the network depth increases. Optimization is generally difficult, but in some cases one can overcome the difficulty by exploiting the structure of the cost function. If that one is continuous, convex, or differentiable, powerful methods are known, such as gradient-based methods including Stochastic Gradient Descent¹ or gradient-free methods including COBYLA or Nelder-Meads^{2,3}.

However, combinatorial optimization, which we focus on in this paper, turns out to be a much more difficult task. The solution space of combinatorial problems is the space of all possible bit-strings of a given length N : $\{0, 1\}^N$. This explains why naive methods – brute-force, linear search – have a cost exponential in N . For discrete binary cost functions, an analogous simplifying feature to continuity or convexity is the recursivity. It allows one to use dynamic programming methods⁴⁻⁶ and changes time complexity into space complexity. Nonetheless, the scope of problems that can be tackled by recursivity is limited. Studies in recent decades have identified extremely challenging combinatorial tasks with problems whose cost function can be expressed as a generalized Ising Hamiltonian⁷⁻¹⁰. The Ising problems have a discrete structure, and when the interaction between non-nearest neighbor spins contributes to the energy spectrum, dynamic programming methods stop working (spin-glass or random energy problems) and optimization becomes extremely challenging. The cost functions in which this complicated structure appears are not limited to physical scenarios but are, in fact, present in many NP / NP-complete problems, such as the knapsack problem or the traveling salesman problem. Moreover, there are cases where there are no explicit form for the function one is trying to optimize. This is a Black-Box scenario and, without much surprise, they are equally challenging. Efficient ways for tackling most general binary cost functions are little known and the best known methods are based on meta-heuristics, such as simulated annealing^{8,11} or genetic algorithms.

In this work, we propose a novel optimization algorithm that can be used to handle combinatorial optimization tasks of high complexity. The method proposed in this paper is based on the principles of compressive sensing¹²⁻¹⁵, statistical compressive learning¹⁶, and Monte-Carlo methods^{17,18}. Compressive sensing is a technique that allows for recovering a high dimensional signal from a small number of linear measurements given that the signal has a sparse representation in a known basis. The recovery procedure explicitly finds the sparse representation from which the high dimensional signal is derived by a basis transformation. The number of measurements needed to recover the total signal scales as a polynomial function of the number of the non-zero entries of the sparse representation, which is much smaller than the dimensionality of the signal. Statistical compressive learning is a relatively new technique for extracting the core features of a distribution from a sample. The sample is mixed by a sketch function and a recovery algorithm applied to the sketching image returns a function that matches the

original distribution better than the estimates based on the sample directly. Finally, Monte Carlo is a known approach that uses stochastic algorithms to determine properties of high dimensional functions intractable by rigorous analytical approaches. The combinatorial optimization method we propose here, Monte Carlo compressive optimization (MCCO), combines attractive features of the three techniques to find the maximum of a function based on a sample drawn from its values. MCCO derives the properties of the maximum value of the function from sketches applied to random samples, even though the maximum may never be met during the sampling process. In fact, the method can be seen as approximating a model whose parameters are learned from the samples and which is carried in the optimization procedure. We provide evidence that the location of the maximum can survive substantial modeling error, and that the choice of the model can be loose. This approach can be useful for a wide range of applications, including reinforcement learning and graph problem optimization.

Despite the exact class of function this method can tackle to perfection remains unknown, we argue that the scope of applicability for our method is large by defining the natural notion of compressible problems. Here, compressibility means that there exists a short formula determining, or well-approximating, all values of the cost function, or one can find an embedding from the compressed description space to the optimization space that maps all the relevant information into the values of the cost function. We show numerical evidence that MCCO is applicable on compressible function, although the compressed representation does not need to be known explicitly. The numerical analysis shows that MCCO, when properly tuned and applied to the compressible combinatorial problems we describe, is able to carry the optimization and is competitive by regard to other more usual methods such as dual annealing (simulated annealing + fast annealing)^{19–21}.

Additionally, we developed an open-source Python library named TrOMA²² (Tractable Optimization Modeling Approach) that integrate the MCCO method described in this work. Most importantly, the library is end-to-end and the API is problem and hardware agnostic. That means anyone can test our method easily on any combinatorial problem, the only requirement is to give a cost function. The library support several optimization backend, including classical dynamic programming and heuristic methods for quantum computer (with a compatibility on real IBM quantum computers).

The paper is structured as follows. The Methods will introduce the MCCO algorithm for Combinatorial Optimization which we follow by giving the description of the class of compressible combinatorial problems. The Numerical Results study this class through numerical simulations. Our results include a comparison between MCCO and dual annealing. The Theoretical Analysis strengthen our results by justifying the algorithm theoretically in general terms. The Discussion will approach applications of the MCCO method to Reinforcement Learning and to the Ising problem. We also discuss the role quantum computers can play in optimization. In the Supplementary Information, we discuss properties of the compressible problem used in the numerical simulations (Section 1) and provide a detailed analysis of an example (Section 4).

Methods

The Monte-Carlo Compressive Optimization algorithm

An optimization problem consists of finding the optimum (maximum or minimum) of a given real-valued function f , called the objective, cost, loss, or reward function. Consider $f : \{0, 1\}^N \rightarrow \mathbb{R}^+$, a cost function over the set of all possible bit-strings of length N . We use indistinctly f to talk about both the function and the discrete probability distribution whose f is the density (after it is normalized). Our goal is to find the best input bit-string x^* that maximizes the function f . We define it as

Problem I $x_I^* = \arg \max f(x).$

Let us introduce a new algorithm for combinatorial optimization. The full scheme can be seen in Figure 1.

Algorithm 1 [Monte Carlo Compressive Optimization]

Inputs: sample size n , threshold parameter t , sketch function Φ , decoding procedure Decode.

alg1

1. Compute a sample of f called Sf . To do so: First, draw a sample $\mathcal{J}$ of n indices from the uniform distribution. Then apply the sampling operator $S_{\mathcal{J}}$ defined as $S_{\mathcal{J}} : f(x) \mapsto f(x)\delta_{x,y \in \mathcal{J}}$ on f , where $\delta_{x,y \in \mathcal{J}}$ is the Kronecker delta for all y in f .
2. Apply a *hard thresholding* operator T_t such that in $T_t Sf$ all values $Sf(x)$ lower than a given t become zero.
3. Apply a *sketching map* (also known as a measurement map) Φ to get $y = \Phi(T_t Sf)$. We call y a *sketch vector*. It contains an empirical estimate of some *generalized moments* of the distribution f .
4. Apply a decoding procedure Decode to y and get $\tilde{f} = \text{Decode}(y)$. The decoding procedure is taken from compressive sensing **greedy methods** (Matching pursuit, Orthogonal Matching Pursuit, ...).
5. The largest value of $\tilde{f}$ is used as the maximum estimate.

Throughout the analysis, the threshold is assumed to lie in the range $0 \leq t \leq f(x_2)$, where $x_2 = \arg \max_{x \neq x^*} f(x)$ is the second maximum of f , so that the gap of f is left unchanged by T_t .

The Monte-Carlo method¹⁷ can be used to approximate generalized moments similar to those from the reference¹⁶. This is a well known result from the probabilistic computation theory (or ergodic theory), about a large number M of samples sampled independently from a probability distribution f ,

$$\lim_{M \rightarrow \infty} \frac{1}{M} \sum_{x_{(i)} \sim f} g(x_{(i)}) = \sum_{x_{(i)}} g(x_{(i)}) f(x_{(i)}) \approx \frac{S}{M} \sum_{x_{(i)} \sim \mathcal{U}} g(x_{(i)}) f(x_{(i)}), \quad (1)$$

where S is the size of the domain. Which means that a moment of a function g of a variable x with respect to a measure $f(x)$ can be approximated by sampling from the uniform distribution $\mathcal{U}$ and computing the value of the density function $f(x)$ in those points.

Regarding the choice of the sketch function, we know from compressive sensing theory that functions Φ that satisfy some properties such as the Restricted Isometry Property (RIP) are good candidates for sketch functions which should preserve important properties of the tested function – in practice, often randomized linear maps are used. Structured deterministic measurement functions can also work¹⁵. For instance in our past works^{23,24}, structured measurement functions with different benefits were used. Moreover, the sketch function does not necessarily need to be linear; generalized moments of the distribution can be of different degrees. Introducing nonlinearities such as a hard thresholding process¹⁵ in practice gives us better performance with respect to random linear map alone. In addition, the choice of the sketch function directly impacts the optimization step within the recovery algorithm. The freedom in the choice of the sketch function allows for flexibility in the algorithm complexity, which can be tailored to match one's available computational resources. Indeed, this turned out to be an essential element of practical implementation (this is important for High Performance Computing and crucial when we consider using quantum computers).

Let us underline the importance of using greedy methods as the matching pursuit rather than ℓ_1 minimization method such as the basis pursuit used in standard compressive sensing. Recovery methods based on ℓ_1 aim to recover the exact sparse signal by putting more emphasis on the constraints. This might be a desirable behavior in compressive sensing, but not in MCCO. When we place more emphasis on the constraints, we learn about the particular features of the selected sample rather than the global features of the distribution. This phenomenon is known in learning theory as overfitting. On the other hand, greedy methods, such as matching pursuit, emphasize the sparsity. These methods allow for finding the most significant element of the tested function that matches the best all constraints. This element often matches the searched maximum line of the distribution.

In the Numerical Results, we use the Orthogonal Matching Pursuit for the decoding procedure and several sketch functions to compute generalized moments. The sketch functions used in this paper are:

Quadruplets interactions

Each line of the sketching matrix is defined by

$$\begin{aligned} \Phi_{x_1, x_2, x_3, x_4}^{i, i+1, i+2, i+3} &= 1_1 \otimes \dots \otimes 1_{i-1} \\ &\otimes (1 - x_1, x_1) \otimes (1 - x_2, x_2) \\ &\otimes (1 - x_3, x_3) \otimes (1 - x_4, x_4) \\ &\otimes 1_{i+5} \otimes \dots \otimes 1_N \end{aligned}$$

for every starting position $1 \leq i \leq N - 3$ and x_i s take all binary values.

Quintuplets interactions

Each line of the sketching matrix is defined by

$$\begin{aligned} \Phi_{x_1, x_2, x_3, x_4, x_5}^{i, i+1, i+2, i+3, i+4} &= 1_1 \otimes \dots \otimes 1_{i-1} \\ &\otimes (1 - x_1, x_1) \otimes (1 - x_2, x_2) \\ &\otimes (1 - x_3, x_3) \otimes (1 - x_4, x_4) \\ &\otimes (1 - x_5, x_5) \otimes 1_{i+6} \\ &\otimes \dots \otimes 1_N \end{aligned}$$

for every starting position $1 \leq i \leq N - 4$ and x_i s take all binary values.

Random sketch A random matrix whose coefficients are drawn from a Gaussian distribution.

Later in the Theoretical Analysis, we use the Matching Pursuit algorithm for the decoding procedure. This defines the following problem for the first crucial step of Matching Pursuit:

Problem II $x_{II}^* = \arg \max \Phi^T \Phi T S f.$

The Class of Compressible Combinatorial Problems

The MCCO algorithm described above is not assumed to solve arbitrary combinatorial problems. However, the scope of its applicability is large and contain classes of natural problems. In this section, we identify a class of objective functions that we call *compressible*. In the Numerical Analysis, we show that MCCO is solid when applied to this class of combinatorial problems and can compete with other methods like dual annealing.

Consider a compressible objective function $f_{\mathcal{R}} : x \mapsto f_{\mathcal{R}}(x)$, where $x \in \{0, 1\}^N$ and $f_{\mathcal{R}}$ is uniquely determined by a set $\mathcal{R}$ of few rules. A rule (r, ω_r) can be defined as a bit-string $r \in \mathcal{R}$ of a given length $k \leq N$ and an associated reward $\omega_r \in \mathbb{R}$. Only if the input bit-string x contains r as a sub-string, the total cost $f_{\mathcal{R}}(x)$ increases by ω_r .

Therefore, for a given input x , the function is defined as:

$$f_{\mathcal{R}}(x) = \sum_{r \in \mathcal{R}} \sum_{i=1}^{N-k+1} \omega_r \delta_{h(x_{i \rightarrow i+k-1}, r), 0} \quad (2)$$

where $x_{i \rightarrow i+k}$ is a k -length sub-string of x starting from the i -th bit, δ is the Kronecker delta, and h is the Hamming distance. From now on, let us use the notation $\delta_r(x_{i \rightarrow i+k-1}) := \delta_{h(x_{i \rightarrow i+k-1}, r), 0}$.

The function is compressible in the way that all the information is contained in a few values and there exists a (non-linear) mapping $F_N : \mathcal{R} \rightarrow \mathbb{R}^{2^N}$ embedding the rules into the optimization space. This mapping is given by the following transform:

$$F_N(r) = \sum_{x \in \{0,1\}^N} \sum_{i=1}^{N-k+1} \delta_r(x_{i \rightarrow i+k-1}) \mathbf{x} \quad \text{eq:transform} \quad (3)$$

where $\mathbf{x}$ is a vector of which the x -th element is 1 and the rest is 0. The transform F_N maps a rule r to a 2^N real value vector that contains the value taken by function $f_{\mathcal{R}}$ on $\{0, 1\}^N$. For more properties of this transform, see the Supplementary Information Section 1. We can define the following vector:

$$f_{\mathcal{R}} = \sum_{r \in \mathcal{R}} \omega_r F_N(r), \quad (4)$$

as a linear combination of the transform of all rules in $\mathcal{R}$ and thus redefine the function $f_{\mathcal{R}}(x)$ as the x -entry of the vector $f_{\mathcal{R}}$.

We now show the complexity of optimizing compressible functions by reducing them to well-known difficult problems. Let us rewrite $f_{\mathcal{R}}$ as a diagonal matrix:

$$\begin{aligned} \mathcal{H} &= \text{diag}(f_{\mathcal{R}}) \\ &= \sum_{r \in \mathcal{R}} \omega_r \text{diag}(F_N(r)) \\ &= \sum_{x \in \{0,1\}^N} \sum_{r \in \mathcal{R}} \sum_{i=1}^{N-k+1} \omega_r \delta_r(x_{i \rightarrow i+k-1}) \text{diag}(\mathbf{x}) \end{aligned}$$

Physicist may not overlook similarities between this diagonal form of the cost function and a more general version of the Ising problem (one in which the value of each ω_r depends on the spin configuration $\omega_r(x)$). Using the conventional mapping from Binary models to Ising Hamiltonians, we have:

$$\mathcal{H} = \sum_{r \in \mathcal{R}} \sum_{i=1}^{N-k+1} \omega_r \delta_r(x_{i \rightarrow i+k-1}) \bigotimes_{j=i}^{i+k} \left(\frac{\mathbb{I} + (-1)^{x_j} \sigma_j^z}{2} \right) \quad (5)$$

where x_i is the i -th bit of x , the tensor product term implicitly contains the identities for subspaces that are not covered by j . Expanding the right hand part, one gets exactly the form of a general Ising Hamiltonian including interactions between

the k -closest spins (tensor product of k terms σ^z with consecutive indices), and non-linear coupling. For more details on the connections with Ising Hamiltonians, refers to the Discussion.

Reduction to the general Ising Hamiltonian makes this optimization problem, in general, NP-complete. This complexity is due to the structure of the reward function $f_{\mathcal{R}}$ that does not have any particular property, such as continuity, convexity, or differentiability, and the solution space is of size 2^N , where N is the length of the input bit-string. A tailored method for such a reward function does not exist and, most likely, it would be approached as optimizing a black box. In that case the state-of-the-art methods rely on meta-heuristics like simulated annealing or quantum annealing.

Numerical Results

In this section, we numerically compare the performances of the Monte-Carlo Compressive Optimization algorithm ^{sec:numeric} when using different sketch functions Φ . We compare the performance of MCCO using each of them together and to a more standard approach: the dual annealing.

Setup : To work with reasonable computational time, we choose a problem of relatively small size, bit-strings of length $N = 12$ (space dimension is 2^N). This also allows for visualization of the problem structure. The case with larger scale and realistic problems will be the subject of separate study. However, the theoretical arguments from the Theoretical Analysis and the tests being currently realized through the TrOMA library, are robust evidence that similar conclusions will still hold on larger cases. The problem of interest is shown in Figure 2 and was randomly drawn from the class of compressible problems described earlier. We recall that these problems belong to a general class of Ising-like cost functions that are complicated to solve and often considered as black-box problems.

We chose a set $\mathcal{R}$ of possible rules described by sequences of four, five, or six bits. The length of each rule, as well as each pattern, are chosen randomly. We compute an estimate for four distinct methods. The dual annealing method is used for reference. We compare with our MCCO algorithm using three distinct sketch functions described previously: Random, Quadruplets, and Quintuplets. The metric used in Figure 3 is the average distance (over the choice of samples) to the real optimum $|f(x^*) - f(\hat{x})|$. We compare this distance for different sample size n going from 50 to 400.

Additionally, Figure 4 shows the distribution of these distances for estimates calculated from each sample. Another interesting metric is the Hamming distance of the estimate string $\hat{x}$ to the optimal one x^* . Indeed, if the algorithm provide candidates within close Hamming distance, a simple post-processing step (like a greedy bit swapping) can allow to find optimum solutions. This Hamming distance distribution for the estimates calculated from each sample can be viewed in Figure 5.

In the case of the dual annealing, we chose the number of iterations to be of the same order as the sample size. Indeed, this is a fair comparison since the number of calls to the cost function f , supposedly costly, will remain the same in both cases. In addition, the annealing method is in fact similar to a sampling method—it can be associated with the Metropolis-Hastings algorithm²⁵ where the sampling domain is reduced at each iteration.

Results interpretation: The results are summarized in Figure 3. First, we notice that both sketch functions, quadruplets, quintuplets, and random, reduce the distance to the optimum in a polynomial way. The quintuplets method performs well compared to the random linear sketch function but also to the annealing. Indeed, for the quintuplet method, the distance to the optimum reduces in a manner similar, or faster, to the annealing method. This is strong evidence that, on this class of compressible cost functions, our method can outperform dual annealing, especially in the useful case of the small sample size solution.

In Figure 4, we show the distribution of the distance between the MCCO estimate and the real optimum. We start with MCCO using the quadruplets and quintuplets sketches and compare them to dual annealing. We can see that quadruplet and quintuplet sketches allow us to successfully recover the global maximum of up to around 58% and 49% cases, respectively. This surpasses the success rate of dual annealing, which only finds the global optimum in around 46% of the cases in this example. Moreover, if MCCO fails to recover the global maximum, the estimate is still a local optimum the value of which is within one σ (STD) interval from the optimal with high probability. Specifically, the probability of finding a local optimum within one σ reaches 87% for MCCO with Quadruplet sketch and 79% for the case of the Quintuplet sketch. Both exceed the value 71% that we find for dual annealing. We conclude that MCCO used with Quadruplet or Quintuplet sketch functions find a good optimum more often than dual annealing on the considered class of problems.

In Figure 5, we summarize the distribution of the Hamming distance between the MCCO estimate and the real optimum. Consider MCCO with the quadruplet and quintuplet sketch functions in relation to dual annealing. We can see that quadruplet and quintuplet based MCCO suggests solutions that lie within a small Hamming distance from the real optimum. For the Quadruplet case, this distance is zero in 58% of the cases, less than or equal to one in 91% of the cases, and less than or equal to two in 99% of the cases. For MCCO with the Quintuplet sketch, this distance is zero in 49% of the cases, less than or equal to one in 85% of the cases, and less than or equal to two in 97% of the cases. In the considered class of problems, this surpasses

dual annealing, which only suggests a solution with the Hamming distance zero in 46% of the cases, less than or equal to one in 81% of the cases, and less than or equal to two in 95% of the cases. We conclude again that MCCO with Quadruplets or Quintuplets sketch functions finds the optimum lying within a close Hamming distance to the global optimum more often than the dual annealing method on this class of problems.

Consider MCCO with the random sketch function. Figure 4 shows that its performance is not better than dual annealing, with a global optimum recovered only 6% of times. Looking solely at the functional distance, one can wonder whether MCCO with the random sketch is better than a random search, since the solution can sometimes lie far from the global optimum. However, looking at the Hamming distance in Figure 5, even when using a random sketch map, we can see that the Hamming distance never drops below half of its maximum. This implies that the MCCO algorithm with any reasonable sketch function does not perform worse than a random guess.

All in all, our numerical experiment show very solid performances for the MCCO method on compressible problems. In addition, the MCCO algorithm may be preferable to other method that would be used in that case as for instance the dual annealing. The open-source TrOMA²² library we developed easily allow anyone to use MCCO and reproduce our results. As the scope of problems our method handle the best is not yet perfectly known, we would like to invite the reader to try our method on their own problem through the library.

Theoretical Analysis

In this section, we give theoretical evidence of the soundness of the MCCO method. In other words, we show that the solution of Problem II is consistent with, or approximately matches, the one of Problem I. sec:guarantees_new

Matrices that preserve the maximum

Let us discuss the properties of appropriate matrices Φ to be used as sketch maps in MCCO (Algorithm 1). It is possible to characterize good sketch matrices depending on the parameters of the problem, e.g. the gap between the two largest values of the tested function. Although definition based on quantitative assumptions remains an open problem, our numerical results demonstrate the existence of suitable sketch matrices. In addition, we give qualitative properties for matrices belonging to this class. Moreover, in the Supplementary Information we discuss details of an example.

Definition 1 (Surrogate and empirical surrogate). Define $G = \Phi^T \Phi$ the so-called Gram matrix. Let g^x denote the x -th column of G , and, for $s \in \{0, 1\}^N$, let $g^x(s)$ and $f(s)$ denote the values taken at s by that column and by f respectively. The *surrogate function* associated with f and Φ is def:surrogate

$$F(x) = 2^{-N} (Gf)_x = \mathbb{E}_s [f(s)g^x(s)], \quad x \in \{0, 1\}^N,$$

where the expectation is taken over s uniformly distributed on $\{0, 1\}^N$. Given a sample $S_n = (s_1, \dots, s_n)$ drawn i.i.d. uniformly on $\{0, 1\}^N$ with replacement, the *empirical surrogate* is

$$\tilde{F}_{S_n}(x) = \frac{1}{n} \sum_{k=1}^n f(s_k)g^x(s_k).$$

By construction $\mathbb{E} [\tilde{F}_{S_n}(x)] = F(x)$ for every x . Since F differs from Gf only by the positive factor 2^{-N} , the two have the same maximizer, and Problem II is the maximization of an empirical surrogate of this kind, built from the sampled and thresholded values of f .

We consider under which conditions the maximum of the objective function f coincides with the maximum of the surrogate $F(x)$. Then, we will give evidence that the maximum can survive under sampling, i.e. $F(x)$ and $\tilde{F}_{S_n}(x)$ have the same maximum. First, let's consider some general properties of G :

- G is a square matrix whose dimensions are $2^N \times 2^N$
- G is symmetric, i.e. $G = G^T$
- Φ can be normalized such that G only has ones on its diagonal

As an additional desirable property to be a suitable matrix, G should behave as a quasi-isometry. For example, this is the case of matrices following the RIP, a well studied property in compressive sensing, or more generally matrices that allow to satisfy the Johnson-Lindenstrauss lemma. Notice that, just like the identity matrix, G already has ones on its diagonal. However, we can allow additional degrees of freedom without changing the position of the maximum of f . In order to achieve

quasi-isometry, the maximum of each row should remain on G 's diagonal, and all off-diagonal elements should be relatively small. These are already the assumed properties for the measurement matrices used in compressive sensing. The off-diagonal elements of G are described by coherence μ of Φ and, in fact, since diagonal elements of G are equal to one, we want μ smaller than 1. However, our numerical results show that too low coherence confers a weak generalization power, which is an issue when we later introduce sampling.

For a structured matrix with nearest-neighbors k -uplets patterns like the one following the algorithm description, the coherence is known and grows as $\mu = \frac{N-k}{N-k+1}$ (see Supplementary Information section 2 for the proof). The quintuplets structured matrix that is used to give Numerical Results has coherence $\mu = \frac{7}{8}$ and the random matrix used in the same section has the average coherence of 0.34.

Other quantitative assumptions on G include that any two rows of G should not be too similar or too different. In fact, problems f whose structure contains a gap between the two largest values should be typically solved under this assumption. Notice that because Φ contains only m rows, the rank of G is smaller or equal to m . Thus, G is not full-rank. Because of the presence of eigenspaces that are mapped to zero, without the above assumption, G tends to close the gap in f . If the rows are too similar, we easily mix input; if they are too different, we amplify the chance to promote a wrong candidate for the maximum. The gap assumption mentioned above is common in optimization algorithms. It is a requirement of many meta-heuristic methods, for instance, in quantum annealing where one has to consider the gap in the optimized Hamiltonian.

An additional assumption on G is that it should be easily decodable. This assumption is not needed for G to preserve the maximum. However, in order for the surrogate F to be a simpler optimization problem than f this property is highly desirable. With that goal in mind, we can artificially limit the choice of G to a class that imply a tractable problem. The structured matrices that we suggested in the Method implement the feasibility assumption. Indeed, the Quadruplet and Quintuplet matrices represent a spin-chain Ising problem and can be solved by dynamic computing, e.g.,⁶. In our previous work²⁴, we also discussed how more complex structured matrices can be decoded using quantum computers (see the Discussion for more details).

Preserving the maximum under sampling

In the previous section, we discussed the existence of sketch matrices which do not change the maximum of the sketched function. In this section, we study how these matrices behave when values of the objective function f are being sampled. Under large enough sampling, we quantify the probability that the maximizer of the empirical surrogate $\tilde{F}_{S_n}$ of Definition 1 coincides with the maximizer of the surrogate F . We also show that we can benefit from applying the thresholding operator T_t in this task.

The following theorem bounds the probability that the maximizer of the surrogate is displaced when the surrogate is estimated from a finite sample.

Theorem 1. Let x^* be the position of the maximum of f , and assume that G preserves this maximum when applied to the full function, i.e. $F(x^*) > F(x)$ for all $x \neq x^*$. For each $x \neq x^*$, let's define the gap random variable Δ_x and it's average Θ_x

$$\Delta_x(s) = f(s) \left(g^{x^*}(s) - g^x(s) \right), \quad \Theta_x = \mathbb{E}_s [\Delta_x(s)] = F(x^*) - F(x) > 0.$$

each with variance σ_x^2 . We also define

$$\Theta_{\min} = \min_{x \neq x^*} \Theta_x = F(x^*) - \max_{x \neq x^*} F(x), \quad \sigma^2 = \max_{x \neq x^*} \text{Var}_s [\Delta_x(s)], \quad M = \max_{x \neq x^*} \|\Delta_x - \Theta_x\|_{\infty}.$$

The following property holds:

$$P \left(\arg \max_x \tilde{F}_{S_n}(x) \neq x^* \right) \leq 2^N \exp \left(\frac{-n\Theta_{\min}^2}{2\sigma^2 + \frac{2}{3}M\Theta_{\min}} \right), \quad \text{eq:sampling_max} \quad (6)$$

and in particular the maximizer is preserved with probability at least $1 - \delta$ whenever

$$n \geq \left(\frac{2\sigma^2}{\Theta_{\min}^2} + \frac{2M}{3\Theta_{\min}} \right) (N \ln 2 + \ln(1/\delta)). \quad \text{eq:sample_size} \quad (7)$$

Proof. The domain $\{0, 1\}^N$ is finite and f is bounded, so each Δ_x is a bounded random variable under uniform sampling. Since the s_k are drawn independently, the variables $\Delta_x(s_1), \dots, \Delta_x(s_n)$ are i.i.d. with common mean $\Theta_x \geq \Theta_{\min} > 0$ and variance $\sigma_x^2 \leq \sigma^2$. The constant M is finite for the same reason, and admits the explicit bound

$$M \leq 2\|f\|_{\infty} \max_{x \neq x^*, s} |g^{x^*}(s) - g^x(s)| \leq 2m\|f\|_{\infty},$$

where m is the number of rows of Φ , since $G = \Phi^T \Phi$ has non-negative integer entries bounded by m . For every $x \neq x^*$, we define the summed random variable $\tilde{\Delta}_x$ associated to a sample set S_n :

$$\tilde{F}_{S_n}(x^*) - \tilde{F}_{S_n}(x) = \frac{1}{n} \sum_{k=1}^n \Delta_x(s_k) =: \tilde{\Delta}_x.$$

Hence x^* is the unique maximizer of $\tilde{F}_{S_n}$ if and only if $\tilde{\Delta}_x > 0$ for all $x \neq x^*$. The equivalence holds since the gap exists for all competitors.

Consequently using the union bound,

$$P\left(\arg \max_x \tilde{F}_{S_n}(x) \neq x^*\right) = P\left(\bigcup_{x \neq x^*} \{\tilde{\Delta}_x \leq 0\}\right) \leq \sum_{x \neq x^*} P(\tilde{\Delta}_x \leq 0).$$

For i.i.d. variables with $\|\Delta_x - \Theta_x\|_\infty \leq M$ and variance σ_x^2 , Bernstein's concentration inequality gives, for all $t > 0$,

$$P(\tilde{\Delta}_x - \Theta_x \leq -t) \leq \exp\left(\frac{-nt^2}{2\sigma_x^2 + \frac{2}{3}Mt}\right).$$

Choosing $t = \Theta_x > 0$, the left-hand side becomes $P(\tilde{\Delta}_x \leq 0)$, so that

$$P(\tilde{\Delta}_x \leq 0) \leq \exp\left(\frac{-n\Theta_x^2}{2\sigma_x^2 + \frac{2}{3}M\Theta_x}\right).$$

Finally, the map $u \mapsto u^2/(2\sigma^2 + \frac{2}{3}Mu)$ is increasing on $u > 0$, so $\Theta_x \geq \Theta_{\min}$ and $\sigma_x^2 \leq \sigma^2$ yield

$$P(\tilde{\Delta}_x \leq 0) \leq \exp\left(\frac{-n\Theta_{\min}^2}{2\sigma^2 + \frac{2}{3}M\Theta_{\min}}\right).$$

Summing over the $2^N - 1 < 2^N$ competitors results in equation (6). Requiring the resulting bound to be at most δ and solving for n gives equation (7). $\square$

Theorem 1 characterizes the behavior of the algorithm under sampling. The probability of success is controlled by the mean, the variance and the range of the random variables Δ_x , and in particular by $\Theta_{\min}$, the gap of the surrogate function. This becomes a statement about the objective itself under the assumptions made above on G : when the sketch acts as a quasi-isometry in the restricted sense of compressive sensing, the surrogate inherits the shape of f , and $\Theta_{\min}$ reflects the gap between the two largest values of the objective function. The dimension of the problem then enters the required sample size only through a factor $N \ln 2$, i.e. logarithmically in the size of the search space, the remaining dependence being carried by $\Theta_{\min}$, σ^2 and M .

Effect of Thresholding

The thresholding step changes the sampled function from f to $T_t f$, where $T_t f(x) = \begin{cases} f(x) & \text{if } f(x) \geq t \\ 0 & \text{otherwise} \end{cases}$. The following proposition shows that, for an admissible threshold, this change leaves both the maximizer and the gap of f untouched.

Proposition 1. Let $f: \{0, 1\}^N \rightarrow \mathbb{R}^+$ have a unique maximizer x^* , let $x_2 = \arg \max_{x \neq x^*} f(x)$ be its second maximum, and let $\Delta = f(x^*) - f(x_2) > 0$ be the gap of f . If $0 \leq t \leq f(x_2)$, then x^* is the unique maximizer of $T_t f$ and the gap of $T_t f$ is exactly Δ . prop:threshold

Proof. Since $f \geq 0$, thresholding only replaces values by zero, so $T_t f(x) \leq f(x)$ for all x . As $t \leq f(x_2) \leq f(x^*)$, both values survive: $T_t f(x^*) = f(x^*)$ and $T_t f(x_2) = f(x_2)$. For any $x \neq x^*$, $T_t f(x) \leq f(x) \leq f(x_2)$. Hence $\max_{x \neq x^*} T_t f(x) = f(x_2) < f(x^*) = T_t f(x^*)$, and the gap of $T_t f$ is $f(x^*) - f(x_2) = \Delta$. $\square$

Since T_t acts pointwise, for a fixed t it commutes with the sampling operator: $T_t S f = S T_t f$. Thresholding the sample is therefore the same as sampling the thresholded function, so the results on sampling apply to $T_t f$ in place of f .

In this analysis, the threshold t is considered fixed, whereas in practice it is chosen from the sampled values. One can consider that a first sample is used to decide t , within the admissible range $0 \leq t \leq f(x_2)$. Once the value of t is fixed, a new sample, independent of the first one, is drawn, so that the resulting empirical surrogate is unbiased for the thresholded function $T_t f$. This is only to make the analysis rigorous; in practice, the samples used before and after fixing t are similar, so the same sample can be used throughout.

cor:threshold

Corollary 1 (of Theorem 1, Thresholded Function). Let $0 \leq t \leq f(x_2)$ and assume that G preserves the maximum of $T_t f$. Then equation (6) holds with x^* the maximizer of f , and with $\Theta_{\min}$, σ^2 and M computed from $T_t f$.

Proof. Theorem 1 only requires the sampled function to be bounded, so it applies to $T_t f$, whose maximizer is x^* by Proposition 1. $\square$

Theorem 2. Theorem to add on thresholding ?

By investigating the effect of thresholding, we conclude that it resulted in a reduction of the variance for the thresholded function. The details of this analysis can be found in the Supplementary Information Section 3. A smaller variance and higher mean values both lead to increasing success probability, justifying the use of the thresholding step in Algorithm 1. ???

Specific case: Preserving the maximum with a structured sketch

We now specialize the analysis to the case where the sketch matrix is structured, e.g. chosen as Quadruplets or Quintuplets, which makes the surrogate of Definition 1 explicit. The generalized moments one estimates are then expectation values of linear functions, i.e. $\hat{a}_S = \mathbb{E}_x[f(x) \prod_{s \in S} x_s]$, so the sketch vector y contains estimates of the Walsh-Hadamard²⁶ coefficients of f , indexed by the set K of monomials carried by Φ . Written in this basis,

$$F(x) = \sum_{S \in K} a_S \prod_{s \in S} x_s, \quad \tilde{F}_{S_n}(x) = \sum_{S \in K} \hat{a}_S \prod_{s \in S} x_s, \quad (8)$$

so the surrogate is the truncation of the Walsh-Hadamard expansion of f to K , and the empirical surrogate is the function actually reconstructed by the algorithm. Problem II is therefore the maximization of $\tilde{F}_{S_n}$ over $\{0, 1\}^n$. The two propositions below, together with the lemma connecting them, make the resulting error budget explicit.

prop:wh_samp

Proposition 2. Given a function f , learning k of its Walsh-Hadamard coefficients, all within given error ε , can be achieved with sample complexity $m = O(\frac{\log k}{\varepsilon^2})$.

Proof. Each coefficient a_S is an expectation of a bounded random variable. By Hoeffding's inequality, $m = O(\frac{1}{\varepsilon^2})$ samples suffice to estimate a single coefficient within error ε . A union bound over k coefficients completes the proof. $\square$

Notice that the sample complexity is independent of n and remains tractable even when the size of the space becomes exponentially large.

lem:linf

Lemma 1. Let K , with $|K| = k$, be the set of monomials carried by Φ . If $|a_S - \hat{a}_S| \leq \varepsilon$ for every $S \in K$, then

$$\|f - \tilde{F}_{S_n}\|_{\infty} \leq k\varepsilon + \sum_{S \notin K} |a_S|. \quad (9)$$

Proof. For any $x \in \{0, 1\}^n$, every monomial satisfies $\prod_{s \in S} x_s \in \{0, 1\}$, hence $|\prod_{s \in S} x_s| \leq 1$. Splitting

$$f - \tilde{F}_{S_n} = \sum_{S \in K} (a_S - \hat{a}_S) \prod_{s \in S} x_s + \sum_{S \notin K} a_S \prod_{s \in S} x_s$$

into the monomials of K , whose coefficients are estimated within ε , and those outside K , which contribute their full value, and applying the triangle inequality gives the result. $\square$

The two terms are the two error sources of the method: $k\varepsilon$ bounds $\|F - \tilde{F}_{S_n}\|_{\infty}$, the sampling error already controlled by Theorem 1, while $\sum_{S \notin K} |a_S|$ equals $\|f - F\|_{\infty}$, the sketching error coming from the choice of K . Which coefficients are missing thus matters only through their ℓ_1 mass, and compressibility of f is precisely the assumption that this mass is small.

prop:gap

Proposition 3. Given a function f , $f(x^*)$ its maximum and $\Delta = f(x^*) - \max_{x \neq x^*} f(x)$ the gap between the two largest values, let $\tilde{F}_{S_n}$ be the empirical surrogate of Definition 1. If the reconstruction error $\|f - \tilde{F}_{S_n}\|_{\infty} < \Delta/2$, the position of the maximum remains unchanged, i.e.

$$\arg \max_x \tilde{F}_{S_n}(x) = \arg \max_x f(x) = x^*. \quad (10)$$

Proof. Assume $\|f - \tilde{F}_{S_n}\|_\infty < \Delta/2$. Then for any $x \neq x^*$:

$$\tilde{F}_{S_n}(x^*) \geq f(x^*) - \frac{\Delta}{2}, \quad \text{eq:bound1} \quad (11)$$

$$\tilde{F}_{S_n}(x) \leq f(x) + \frac{\Delta}{2} \leq f(x^*) - \Delta + \frac{\Delta}{2} = f(x^*) - \frac{\Delta}{2}. \quad \text{eq:bound2} \quad (12)$$

Combining (11) and (12) we have $\tilde{F}_{S_n}(x^*) \geq \tilde{F}_{S_n}(x)$ for all $x \neq x^*$. In other words, the argmax is preserved. $\square$

Combining Proposition 2 and Proposition 3 through Lemma 1, the maximum is preserved as soon as $k\varepsilon + \sum_{S \notin K} |a_S| < \Delta/2$. Taking $\varepsilon = \Delta/(4k)$, this holds whenever the discarded coefficients carry a mass smaller than $\Delta/4$, at a sample complexity $m = O(k^2 \log k / \Delta^2)$, independent of n . The condition of Proposition 3 is sufficient but not necessary, such that the maximum can survive even larger perturbations.

Discussion on applications and quantum computers

Compressible combinatorial problems naturally show up in many fields. To see that, let us discuss learning an optimal policy in Reinforcement Learning, and finding the ground state of an Ising Hamiltonian. Reinforcement Learning is a crucial technique in today's machine learning due to its central role in several applications, such as optimal control theory, robotics, or game theory. Among the approaches to tackle it, Monte-Carlo methods solve the task based on complete returns obtained from full episodes; the name comes from the fact that one is sampling through the space of complete returns. Consider the following "game": at each step, the agent has to choose between two possible actions labeled 0 and 1, and only after one complete episode, which consists of a sequence of N actions, a total reward is granted. The goal is to find a nearly optimal policy maximizing the total reward. Since one has only access to the final returns, with the reward mechanism in each episode acting like a black-box, the use of Monte-Carlo methods seems reasonable in this context. Thus, this reinforcement learning scenario exhibits a problem structure similar to the combinatorial optimization of a compressible function, and our approach can be used here as a competitive method in a context where policy networks could not be exploited to their fullest potential.

Another such problem is finding the ground state of an Ising Hamiltonian. Generalized Ising models are known to be intractable for classical computers. A random energy Ising is given by:

$$\mathcal{H}(\sigma) = \sum_{\sigma_s \in P} \omega_s \sigma_s \quad (13)$$

where σ_s are Pauli-Z strings within the Pauli group P of given length s . We already made a parallel between such generalized Ising Hamiltonians and our cost function; our method is a new promising approach to tackle the problem of finding their ground state, as no efficient classical methods are known (and there won't be if $P \neq NP$).

Quantum computers also seem to have a crucial role to play here, especially since so many cost functions can be mapped to Ising-like Hamiltonians. Even though they might not be able to solve the optimization problem in polynomial time (if $NP \not\subset BQP$)²⁷, they might provide good heuristics to tackle those kinds of problems (see quantum annealing⁹ and QAOA²⁸). In our previous work²⁴, using structured maps similar to the Quadruplets and Quintuplets defined in the Methods, the optimization step in matching pursuit reduced to finding the ground state of an Ising Hamiltonian. For distant-pair patterns, this reduces to a spin-glass problem and can be approached using QAOA or quantum annealing, a priori faster than using known classical algorithms. More generally, when applying the Monte-Carlo Compressive Optimization algorithm with a sketch function made of structured patterns, we map the complex Ising problem into a Ising problem where its optima is preserved. The mapping is done using a sample of the solution space, hence it is computable in a tractable time.

Another particularly important merit of our method is that it opens the way to optimization being carried out on specific devices like digital annealers or quantum computers. Such devices often have limited connectivity or considerable error rates, and our method allows the model to be adapted to fit these restrictions. In those two previous works^{29,30}, the authors tried a similar approach by learning a surrogate model through sampling so that the optimization could be run on a quantum annealer. Extending the scope of problem that can benefit from these devices is an active track of research, and our method can be seen as a refined version of these two works, where we allow more general models and give sound theoretical evidence.

References

1. Robbins, H. & Monro, S. A Stochastic Approximation Method. *The Annals Math. Stat.* **22**, 400–407, DOI: [10.1214/aoms/1177729586](https://doi.org/10.1214/aoms/1177729586) (1951). Publisher: Institute of Mathematical Statistics.
2. Nelder, J. A. & Mead, R. A Simplex Method for Function Minimization. *The Comput. J.* **7**, 308–313, DOI: [10.1093/comjnl/7.4.308](https://doi.org/10.1093/comjnl/7.4.308) (1965). _eprint: <https://academic.oup.com/comjnl/article-pdf/7/4/308/1013182/7-4-308.pdf>.

3. Powell, M. J. D. A Direct Search Optimization Method That Models the Objective and Constraint Functions by Linear Interpolation. In Gomez, S. & Hennart, J.-P. (eds.) *Advances in Optimization and Numerical Analysis*, 51–67, DOI: [10.1007/978-94-015-8330-5_4](https://doi.org/10.1007/978-94-015-8330-5_4) (Springer Netherlands, Dordrecht, 1994).
4. Bellman, R. The theory of dynamic programming. *Bull. Am. Math. Soc.* **60**, 503–515, DOI: [10.1090/S0002-9904-1954-09848-8](https://doi.org/10.1090/S0002-9904-1954-09848-8) (1954).
5. Barahona, F. On the computational complexity of Ising spin glass models. *J. Phys. A: Math. Gen.* **15**, 3241, DOI: [10.1088/0305-4470/15/10/028](https://doi.org/10.1088/0305-4470/15/10/028) (1982).
6. Schuch, N. & Cirac, J. I. Matrix Product State and mean field solutions for one-dimensional systems can be found efficiently. *Phys. Rev. A* **82**, 012314, DOI: [10.1103/PhysRevA.82.012314](https://doi.org/10.1103/PhysRevA.82.012314) (2010).
7. Lucas, A. Ising formulations of many NP problems. *Front. Phys.* **2**, DOI: [10.3389/fphy.2014.00005](https://doi.org/10.3389/fphy.2014.00005) (2014). ArXiv:1302.5843 [cond-mat].
8. Kirkpatrick, S., Gelatt, C. D. & Vecchi, M. P. Optimization by Simulated Annealing. *Science* **220**, 671–679 (1983).
9. Kadowaki, T. & Nishimori, H. Quantum Annealing in the Transverse Ising Model. *Phys. Rev. E* **58**, 5355–5363, DOI: [10.1103/PhysRevE.58.5355](https://doi.org/10.1103/PhysRevE.58.5355) (1998).
10. Ohzeki, M. & Nishimori, H. Quantum annealing: An introduction and new developments, DOI: [10.48550/arXiv.1006.1696](https://doi.org/10.48550/arXiv.1006.1696) (2010). ArXiv:1006.1696 [cond-mat].
11. Delahaye, D., Chaimatanan, S. & Mongeau, M. Simulated Annealing: From Basics to Applications. In Gendreau, M. & Potvin, J.-Y. (eds.) *Handbook of Metaheuristics*, 1–35, DOI: [10.1007/978-3-319-91086-4_1](https://doi.org/10.1007/978-3-319-91086-4_1) (Springer International Publishing, Cham, 2019).
12. Candes, E., Romberg, J. & Tao, T. Robust uncertainty principles: exact signal reconstruction from highly incomplete frequency information. *IEEE Transactions on Inf. Theory* **52**, 489–509, DOI: [10.1109/TIT.2005.862083](https://doi.org/10.1109/TIT.2005.862083) (2006).
13. Donoho, D. Compressed sensing. *IEEE Transactions on Inf. Theory* **52**, 1289–1306, DOI: [10.1109/TIT.2006.871582](https://doi.org/10.1109/TIT.2006.871582) (2006).
14. Baraniuk, R., Davenport, M., DeVore, R. & Wakin, M. A Simple Proof of the Restricted Isometry Property for Random Matrices. *Constr. Approx.* **28**, 253–263, DOI: [10.1007/s00365-007-9003-x](https://doi.org/10.1007/s00365-007-9003-x) (2008).
15. Foucart, S. & Rauhut, H. Basic Algorithms. In Foucart, S. & Rauhut, H. (eds.) *A Mathematical Introduction to Compressive Sensing*, 61–75, DOI: [10.1007/978-0-8176-4948-7_3](https://doi.org/10.1007/978-0-8176-4948-7_3) (Springer, New York, NY, 2013).
16. Gribonval, R., Blanchard, G., Keriven, N. & Traonmilin, Y. Compressive Statistical Learning with Random Feature Moments, DOI: [10.48550/arXiv.1706.07180](https://doi.org/10.48550/arXiv.1706.07180) (2021). ArXiv:1706.07180 [stat].
17. Metropolis, N. & Ulam, S. The Monte Carlo Method. *J. Am. Stat. Assoc.* **44**, 335–341, DOI: [10.2307/2280232](https://doi.org/10.2307/2280232) (1949). Publisher: [American Statistical Association, Taylor & Francis, Ltd.].
18. Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H. & Teller, E. Equation of State Calculations by Fast Computing Machines. *The J. Chem. Phys.* **21**, 1087–1092, DOI: [10.1063/1.1699114](https://doi.org/10.1063/1.1699114) (1953).
19. Xiang, Y., Sun, D. Y., Fan, W. & Gong, X. G. Generalized simulated annealing algorithm and its application to the Thomson model. *Phys. Lett. A* **233**, 216–220, DOI: [https://doi.org/10.1016/S0375-9601\(97\)00474-X](https://doi.org/10.1016/S0375-9601(97)00474-X) (1997).
20. Tsallis, C. Possible generalization of Boltzmann-Gibbs statistics. *J. Stat. Phys.* **52**, 479–487, DOI: [10.1007/BF01016429](https://doi.org/10.1007/BF01016429) (1988).
21. Tsallis, C. & Stariolo, D. A. Generalized simulated annealing. *Phys. A: Stat. Mech. its Appl.* **233**, 395–406, DOI: [https://doi.org/10.1016/S0378-4371\(96\)00271-3](https://doi.org/10.1016/S0378-4371(96)00271-3) (1996).
22. baptistechev/TrOMA. Original-date: 2026-03-17T02:25:23Z.
23. Jacob, K. V., Kaur, E., Roga, W. & Takeoka, M. Franck-Condon factors via compressive sensing. *Phys. Rev. A* **102**, 032403, DOI: [10.1103/PhysRevA.102.032403](https://doi.org/10.1103/PhysRevA.102.032403) (2020).
24. Chevalier, B., Roga, W. & Takeoka, M. Compressed sensing enhanced by a quantum approximate optimization algorithm. *Phys. Rev. A* **110**, 062410, DOI: [10.1103/PhysRevA.110.062410](https://doi.org/10.1103/PhysRevA.110.062410) (2024). Publisher: American Physical Society.
25. Hastings, W. K. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika* **57**, 97–109, DOI: [10.1093/biomet/57.1.97](https://doi.org/10.1093/biomet/57.1.97) (1970).
26. O'Donnell, R. Boolean Functions and the Fourier Expansion. In Press, C. U. (ed.) *Analysis of Boolean Functions*, 19–42 (Cambridge University Press, University of Cambridge, 2014). ArXiv:2105:10386.

27. Aaronson, S. NP-complete Problems and Physical Reality, DOI: [10.48550/arXiv.quant-ph/0502072](https://arxiv.org/abs/10.48550/arXiv.quant-ph/0502072) (2005). ArXiv:quant-ph/0502072.
28. Farhi, E., Goldstone, J. & Gutmann, S. A Quantum Approximate Optimization Algorithm, DOI: [10.48550/arXiv.1411.4028](https://arxiv.org/abs/10.48550/arXiv.1411.4028) (2014). ArXiv:1411.4028 [quant-ph].
29. Seki, Y., Nakada, H. & Tanaka, S. Initialization Method for Factorization Machine Based on Low-Rank Approximation for Constructing a Corrected Approximate Ising Model (2024).
30. Aoki, S. *et al.* Formulation of Correction Term in QUBO Form for Phase-Field Model. *Int. J. for Numer. Methods Eng.* **126**, e70019, DOI: [10.1002/nme.70019](https://doi.org/10.1002/nme.70019) (2025). _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/nme.70019>.

Acknowledgements

This work was supported by JST Moonshot R&D, Grant No. JPMJMS226C and Grant No. JPMJMS2061, JST ASPIRE, Grant No. JPMJAP2427, JST COI-NEXT Grant No. JPMJPF2221, JST SPRING Grant No. JPMJSP2123.

Author contributions statement

B.C. conceived the main algorithm and the numerical experiments. B.C., S.Y. and W.R. equally participated in designing the theoretical foundations and proofs. B.C. wrote the majority of the manuscript. W.R. and M.T. supervised the study and provided guidance on the ideas presented in this work. All authors reviewed the manuscript.

Data Availability

The code used to produce the numerical results presented in this work is available as part of the open-source TrOMA library at <https://github.com/baptistechev/TrOMA>.

Additional information

The authors declare no competing interests.

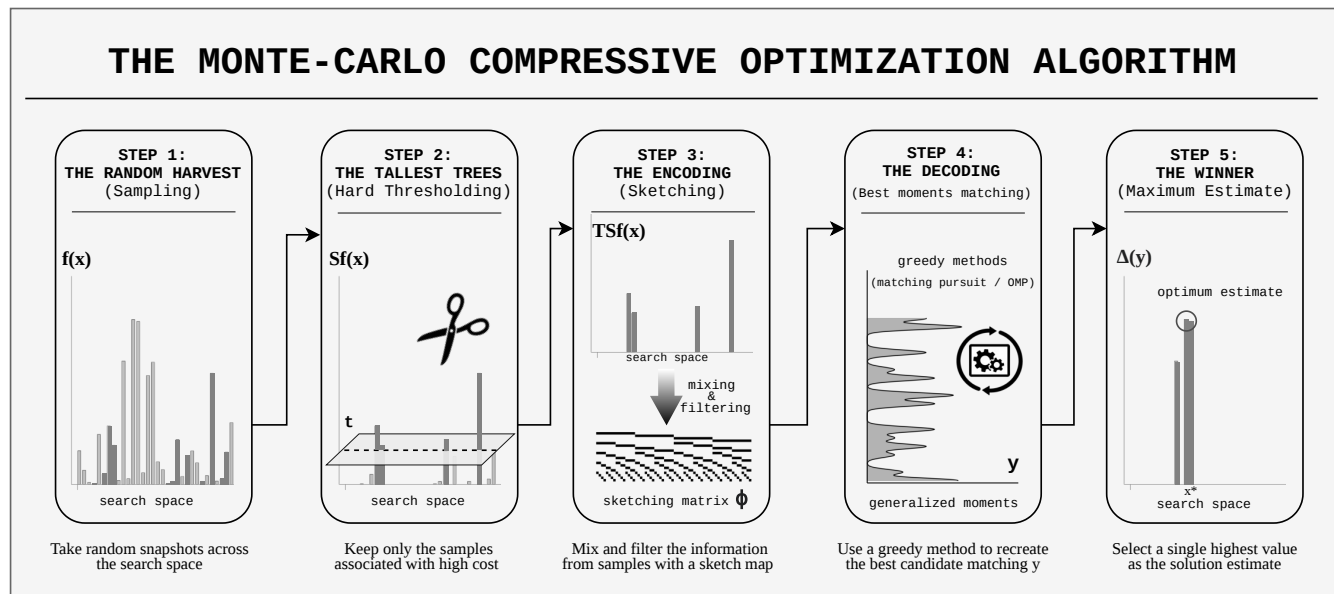


Figure 1. Schematic representation of Algorithm 1: the Monte Carlo Compressive Optimization. The algorithm proceeds through five stages: (1) Sampling of the search space; (2) Hard thresholding to isolate high-value samples; (3) Sketching where survivor samples are compressed into a compact sketch vector y ; (4) Decoding via greedy reconstruction (e.g., OMP); and (5) Estimation of the global maximum from the reconstructed vector.

fig:scheme

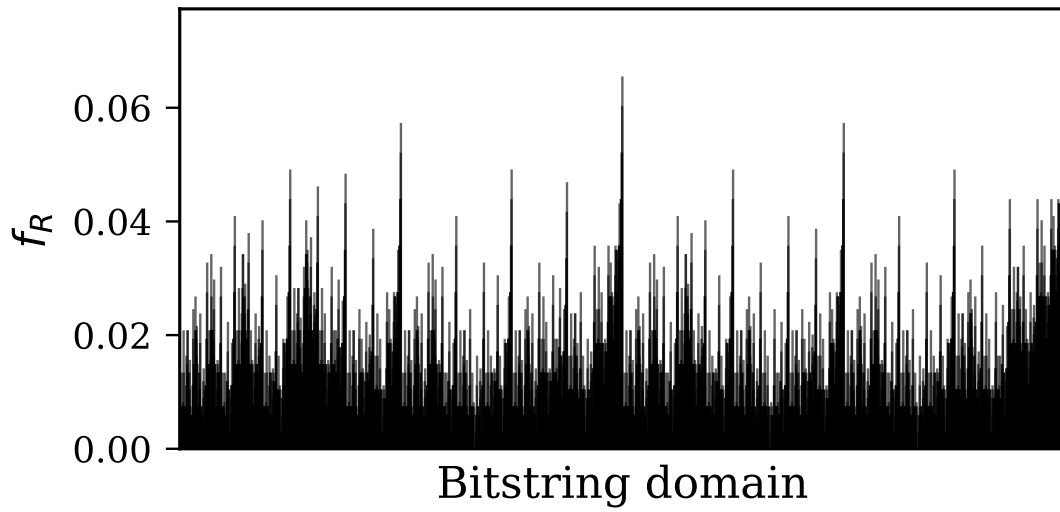


Figure 2. Example of $f_{\mathcal{R}}$. On the x axis, we list all binary sequences. Their respective rewards are shown on the y axis. fig:spectrum

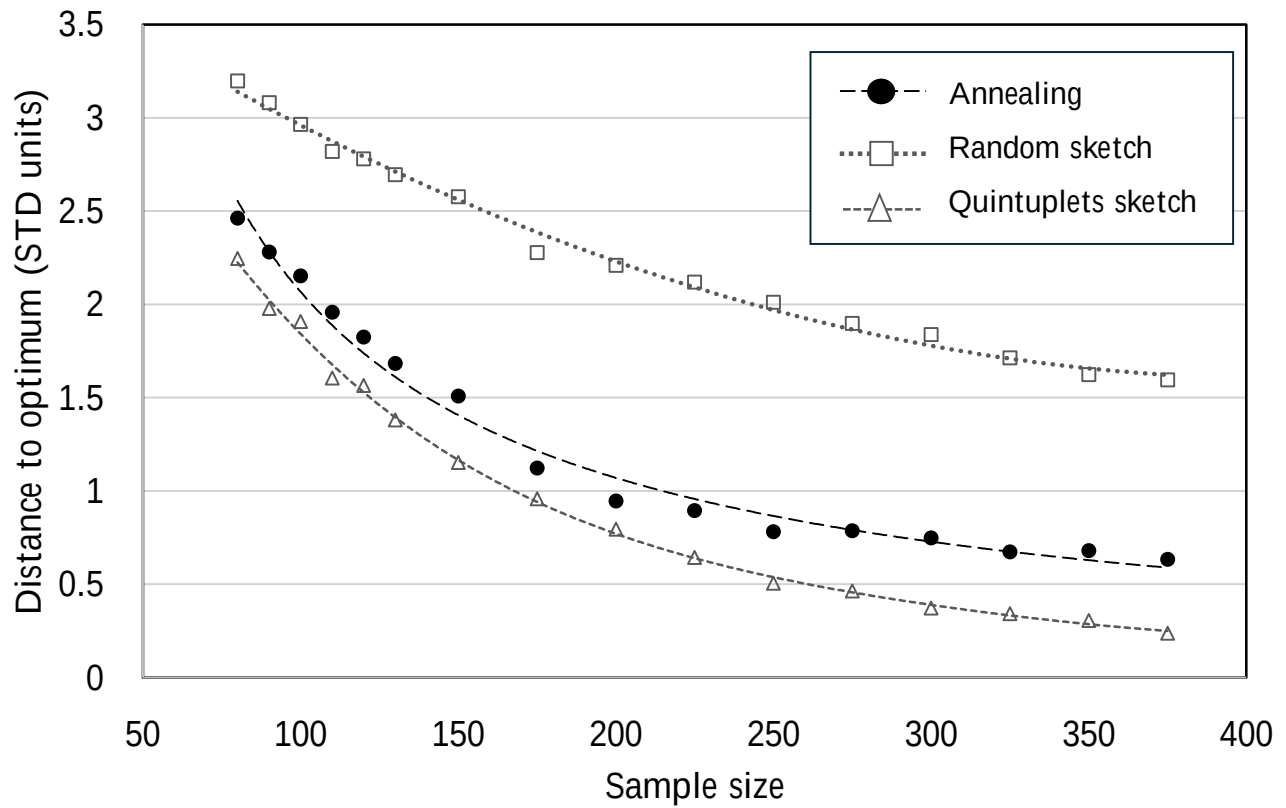


Figure 3. Distance to the optimum when the sample size increases. Comparison of dual annealing (circle) with our method using two different sketch functions: random sketch (square) and quintuplets sketch (triangle). Each point is average over 10000 samples. Units of the y axis are number of σ where σ is the standard deviation of $f_{\mathcal{R}}$. fig:res

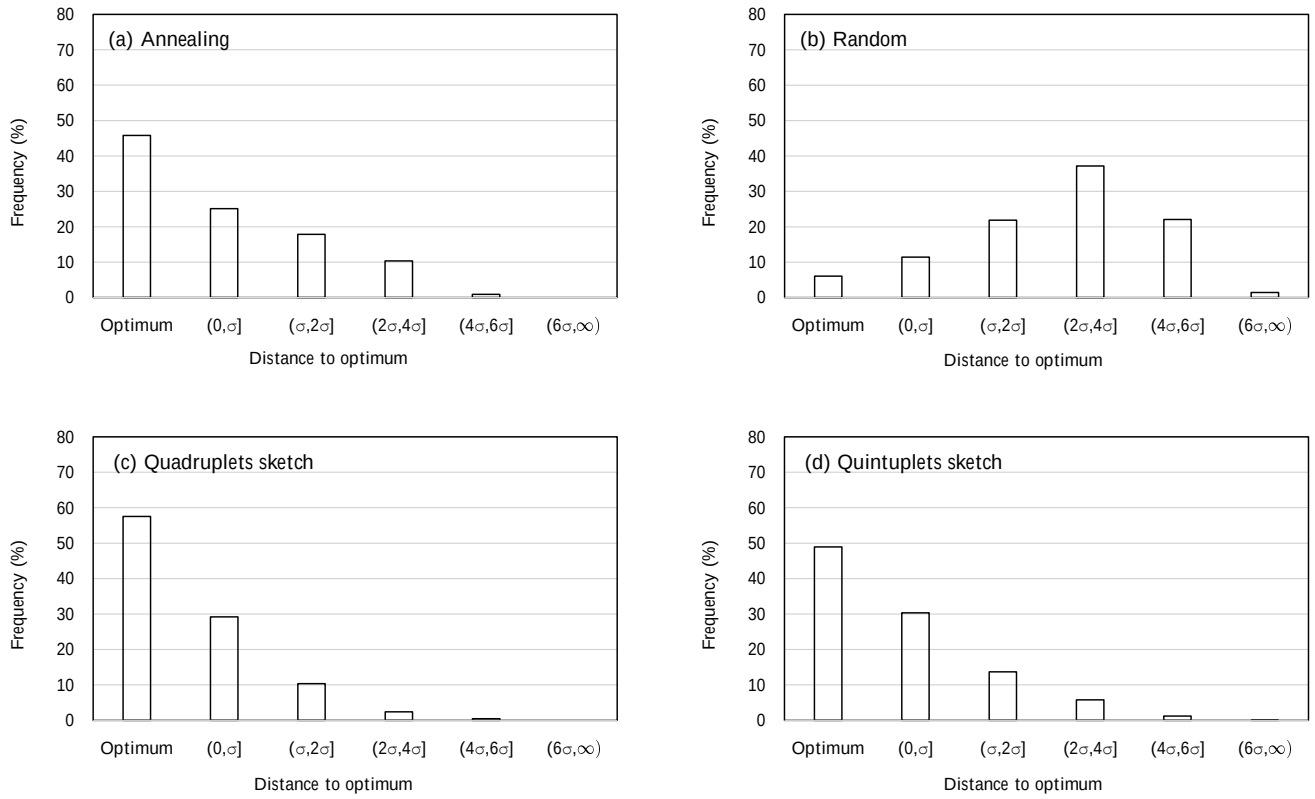


Figure 4. Distribution of the distances $|f(x^*) - f(\hat{x})|$ between estimate solutions $f(\hat{x})$ and the optimum $f(x^*)$. On each plot, the percentage of estimate solutions that lies in given distance interval. The methods used are: (a) annealing and (b)(c)(d) Monte-Carlo Compressive Optimization with different sketch functions. $N = 12$ and $n = 250$.

fig:distance_report

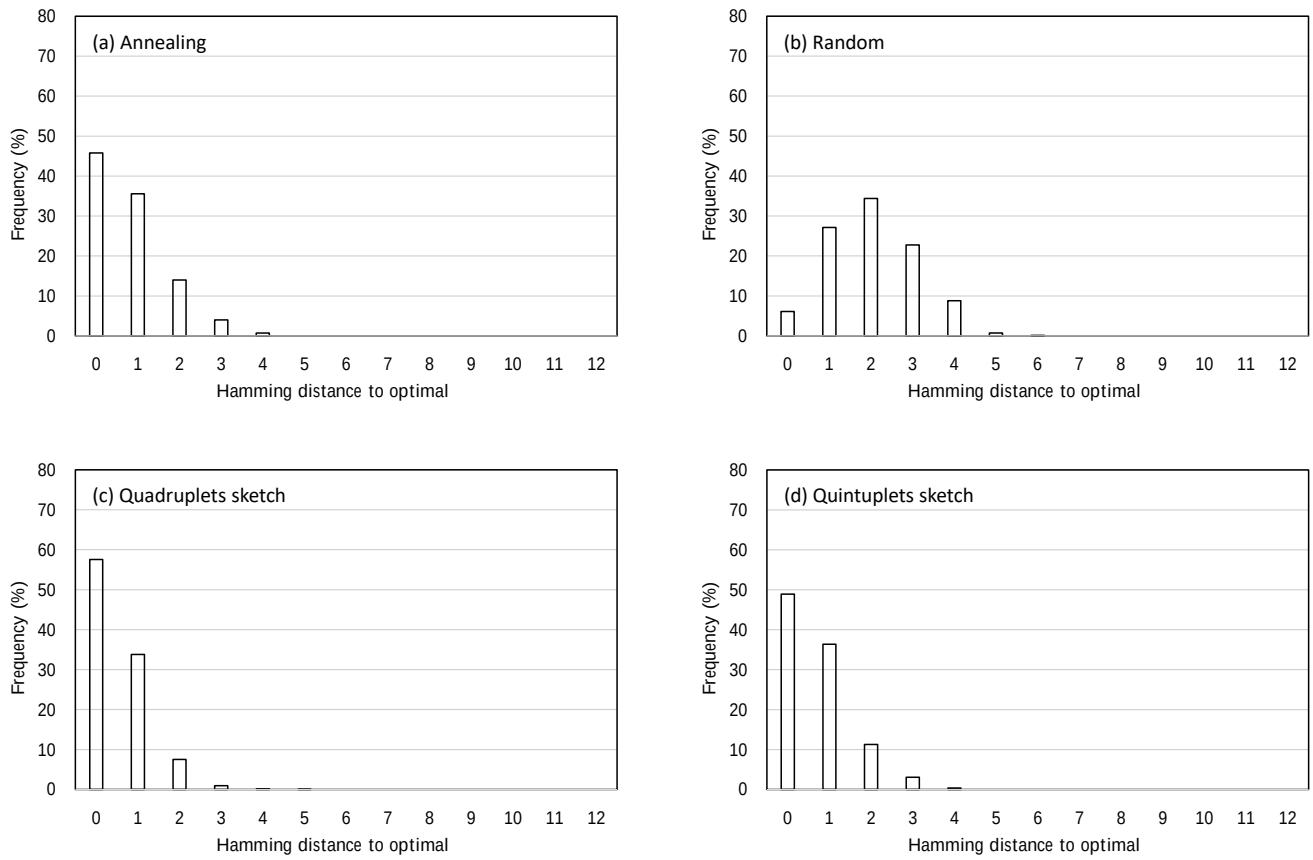


Figure 5. Distribution of the Hamming distance between estimate solutions x and the optimum x^* . On each plot, the percentage of estimate solutions for a given hamming distance. The methods used are: (a) annealing and (b)(c)(d) Monte-Carlo Compressive Optimization with different sketch functions. $N = 12$ and $n = 250$.

fig:ham_repart